

Prozessdokumentation — Website-Migration mit Claude Code

Projekt	KI Beratung Hessen (KIBH) — WordPress/Elementor → statische Website
Bearbeiter	Hari
Projektverantwortlicher	Willy Li
Zeitraum	23.08. – 10.09.2026 · 17 Arbeitstage · 86 Commits
Alt (WordPress)	https://kiberatunghessen.com
Neu (statisch)	https://kibh-webseite.vercel.app

Format nach Vorgabe: Raw Notes, Bullet Points, Snippets. Keine Fließtext-Kapitel.

Zahlenbasis. Alle Messwerte stammen aus dem Projekt selbst: Sitzungstranskripte (275 Einträge), `git log`, `PROGRESS.md`, `docs/arbeitsbericht.md`, `docs/performance-baseline.md` und zwei Lighthouse-Läufen vom 10.09.2026. Geschätzte Werte sind als solche gekennzeichnet.

Lesehilfe zu den Markierungen

- [korrigiert: ...] — sachlicher Fehler in meiner ursprünglichen Notiz, korrigiert und begründet
- [gemessen: ...] — meine Schätzung neben dem tatsächlich gemessenen Wert
- Noch offen** — Frage aus Willys Briefing, die dieser Abschnitt noch nicht beantwortet. Bewusst nicht erfunden.
- Vorschlag** — Punkt, den nicht ich, sondern Claude ergänzt hat

1. Setup & Baseline

Umgebung

Betriebssystem	Windows 11 Pro — grundsätzlich egal, Linux/macOS gehen genauso
Terminal	PowerShell
Editor	VS Code
Node.js	v24.19.0, npm 11.17.0 [gemessen am 10.09.2026]
Claude Code	CLI im Terminal, nicht die Desktop-App — Version 2.1.267
Git / Hosting	Git + GitHub, Deployment auf Vercel

- Diese Einstellungen gelten für **Website-Migration + kleine Verbesserungen**.
- Zielbild: statische Website, höchstens minimale Backend-Teile (z. B. Terminbuchung für KIBH) — und die erst ganz am Schluss.

Claude Code CLI — Installation

Läuft schnell, wenn man schon Ahnung hat. Sonst Schritt für Schritt:

1. **Node.js prüfen** — Laufzeitumgebung, mit der JavaScript-Programme außerhalb eines Browsers laufen. Claude Code wird damit betrieben. `node --version` und `npm --version`. Kommen zwei Versionsnummern → installiert. Kommt ein Fehler → aktuelle Version von <https://nodejs.org/en/download> holen, installieren, danach nochmal prüfen.
 2. **Claude Code installieren und verifizieren:** `npm install -g @anthropic-ai/claude-code`, danach `claude --version`.
 3. **Projektordner anlegen**, in dem die gesamte Entwicklung passiert.
 4. **PowerShell in diesem Ordner öffnen**, dann `claude` starten.
 5. **Einloggen** mit einem Account, der Claude Code unterstützt (Pro oder Max).
- Authentifizierungs- oder Pfadprobleme bei der Claude-Code-Einrichtung selbst: keine.
 - **Aber:** ein Pfadproblem später im Projekt — `python.exe` liegt nicht im PowerShell-PATH. Konsequenz: Python-Skripte laufen über das Bash-Tool. Steht als Dauerregel in `PROGRESS.md`.

Voreinstellungen

- **CLAUDE.md anlegen** und Ziel, Aufgabe und Projektregeln hineinschreiben → Claude hat vor **jeder** Session Kontext, ohne dass ich ihn erneut erkläre. Ein ausgefülltes Template dafür liegt bei mir bereit.
- **Skills installieren.**

[korrigiert: mein Befehl „`npx skills add https://github.com/anthropics/skills --skill webapp-testing`“ war falsch — so wurde es nicht installiert und diesen Befehl gibt es nicht. Tatsächlicher Ablauf laut Transkript vom 23.08.:]

- **frontend-design** — liegt im offiziellen Marketplace, also über den eingebauten Befehl:

```
/plugin install frontend-design@claude-plugins-official
```

Zweck: drückt Claude weg vom generischen KI-Look („Inter font + purple gradient“). Gut zu haben, im Projekt aber kaum genutzt.

- **webapp-testing** — liegt **nicht** im Marketplace. Claude hat `SKILL.md`, `LICENSE.txt`, `scripts/with_server.py` und `examples/` aus `anthropics/skills` nach `~/.claude/skills/webapp-testing/` kopiert, also als User-Level-Skill (nicht projektlokal unter `.claude/skills`). Quelle: <https://github.com/anthropics/skills/tree/main/skills/webapp-testing> Zweck: Claude steuert einen echten Browser — aufrufen, scrollen, messen, Screenshots. Damit prüft er sein Ergebnis selbst, bevor er es mir vorlegt.

[Präzisierung: die Screenshots landen in `tests/screenshots/`, weil unser eigenes Prüfskript `tests/qa.py` sie dort ablegt — nicht weil der Skill diesen Pfad vorgibt.]

- **Wichtigste Einzelentscheidung des Projekts.** Ohne `webapp-testing` kann Claude die Originalseite nur als **Quelltext** lesen. Mit ihm kann er sie **ansehen**. Genau daran hing der teuerste Fehler des Projekts (Abschnitt 4).
- Bilder nach `assets/img/` legen.
- Ordner als GitHub-Repository initialisieren.

Tech-Stack

- **Statisches HTML/CSS/JS. Kein Framework, kein Bundler, null Abhängigkeiten** (`package.json` hat keinen `dependencies` -Block).
- Node.js nur für ein 141-zeiliges Build-Skript (`src/` → `dist/`).
- Python nur für Hilfswerkzeuge (Playwright-Tests, WebP-Kodierung, Freistellen).

Initialer Stand (visuelle Baseline)

- Ursprünglich habe ich **keine** Screenshots gemacht — die URL hat mir gereicht.
- **Nachgeholt am 10.09.:** vollständige Baseline in `docs/baseline/` — alle 10 Seiten der alten WordPress-Site, jeweils Desktop (1440 px) und Handy (390 px), Vollseite. Daneben dieselben Seiten des Neubaus als direkte Vorher/Nachher-Paare. Dateischema: `<seite>__<viewport>__alt.jpg / __neu.jpg` .
- **Merke fürs nächste Projekt:** Baseline **vor** der ersten Codezeile aufnehmen. Sie kostet zehn Minuten und ist später das einzige belastbare „Vorher“.

Modelle, Effort, Usage, Modus

- Claude hat mehrere Modelle. **Nicht immer das stärkste nehmen.** Bild: Modelle sind Mitarbeiter — schwere Aufgaben gehen an die erfahrenen Kollegen.
 - **Sonnet** — kleine Fragen, Meinung einholen, Kleinkram.
 - **Opus** — Entwicklung und Planung, alles was Nachdenken braucht.

- **Effort-Stufe:** mindestens *High*. Für ernstere Sachen *xHigh*, für lange und wichtige Arbeit *Max*.

[Hinweis: die Multiplikatoren, die ich notiert hatte (Opus „2×“, Max Effort „3,5×“), sind meine eigene Beobachtung aus der Nutzungsanzeige und nicht offiziell belegt. Die Richtung stimmt — stärkeres Modell und höherer Effort

- verbrauchen deutlich mehr Kontingent —, die genauen Faktoren sollte man nicht als Fakt verkaufen.]
- **Kontingent:** mit Claude Pro (20 €/Monat), Opus auf Max Effort, ca. **2,5–3 h** Arbeit bis zum Limit, danach 5 h Wartezeit. [gemessen: im Projekt dreimal mitten in einer laufenden Aufgabe eingeschlagen — siehe Abschnitt 4]
- **Modi** — für uns zwei relevant:
 - **Plan-Modus** — beim Planen. Claude darf nichts ändern oder implementieren.
 - **Auto-Modus** — nach der Planung, bei der Umsetzung. Claude darf Dateien bearbeiten.

Session und Projekt

- Session beenden: `/exit` .
- `/compact` fasst den bisherigen Verlauf zusammen → am nächsten Tag geht es mit weniger Tokens weiter.

[korrigiert: ich hatte notiert „Claude macht `/compact` vor jeder Session“. Das stimmt nicht. `/compact` läuft entweder manuell auf Zuruf oder automatisch **während** einer Session, sobald das Kontextfenster voll ist — nicht vorher. Im Projekt ist das 11-mal passiert.]
- **Faustregel: eine Session = ein Thema.** [In diesem Projekt habe ich mich selbst nicht daran gehalten.]
- Ein Projekt (der angelegte Ordner) besteht aus mehreren Sessions.

Prompts mit KI-Hilfe schreiben

- **Wichtig:** Prompts für Claude Code mit einer zweiten KI vorbereiten (Claude Chat, ChatGPT).

- Ablauf: Ich habe die Idee → gebe der KI **allen** Kontext und alle Informationen → lasse den Prompt schreiben → **jedes Wort selbst prüfen** → erst dann in Claude Code einfügen.
- Ergebnis: deutlich genauere Aufträge und weniger Nacharbeit.

Noch offen (Abschnitt 1)

- „Warum eignet sich dieser Tech-Stack für den Prompting-Workflow?“ — Willy fragt das ausdrücklich; meine Notiz nennt nur *welchen* Stack ich gewählt habe, nicht *warum* er zum Prompten passt. Antwort steht noch aus.
-

2. Migration & Core Build

Der Arbeitszyklus (gilt für A und B)

Auftrag → Ausführung → Sichtprüfung im Browser (erst Claude selbst, dann ich) → ggf. Korrektur — und das in der Schleife.

- Klare Phasen wie „Struktur aufbauen → Design/CSS nachbauen → Inhalte befüllen“, wie bei klassischer Entwicklung mit echten Entwicklern, wurden hier **nicht** eingesetzt.
- Bei **großen** Aufträgen (erster Auftrag, neue Features): Prompt mit KI-Hilfe erzeugen.
- Bei **kleinen** Sachen (ein Fehler, eine Farbe): direkt tippen.

[gemessen: die Schleife lief rund 40-mal. Auf jeden großen Auftragsprompt kamen etwa drei kurze

- Korrekturanweisungen — 26 lange gegen 81 kurze. **Kein einziger großer Auftrag lief ohne Nachbesserung durch.**]

A — Wie ich vorgegangen bin

#	Schritt	meine Notiz	gemessen
1	Erster Auftrag: Claude liest <code>CLAUDE.md</code> , dann Planung	1,5 h	2,5 h [Setup 1 h + Planung 1,5 h]
2	Bestandsaufnahme der alten Website durch Claude	1 h	1 h
3	1:1-Nachbau aller Seiten + erstes Deployment auf Vercel	20 min	20 min + 40 min Deployment
4	Neues Feature: Deep Reading Engine (DRE)	3 h	~2 h
5	Korrektur	3 h	~3 h [Regelklärung 1 h + 1:1-Wendepunkt 2 h]
6	Seite-für-Seite-Vergleich mit dem Original (Struktur, Inhalt, Animation) + Korrektur	7 h	~10 h
7	Auftrag „Design deutlich verbessern“ → Fehlschlag, verworfen	1,5 h	1,5 h
8	Qualitätspässe (Mobile, Kleinigkeiten)	6 h	~6 h
9	Abgabe + erste Rückmeldung von Willy → Änderungen in einem Batch	2,5 h	~2,5 h
10	Letzte Assets und Abschluss	2,5 h	~2,5 h

[ergänzt aus git log – diese Schritte fehlten in meiner Liste:]

#	Schritt	Datum	Commits	gemessen
11	Prozessdokumentation Phase 3 geschrieben	08.09.	338b4e5	0 h 33
12	Willys zweiter Batch: DRE-Textkorrekturen, zwei Rollen + Foto, Volker-Karte fertiggestellt, Wortumbruch-Bug, Ergebnisse-Dropdown entfernt, Logo-Größen, Sektion verkleinert	09.09.	3d86bbb , 00adb00	1 h 20
13	Mobile-Audit über 12 Seiten × 6 Breiten + Typografie für schmale Displays	10.09.	2127bd4 , 22fe61d	0 h 33

- **Gesamt gemessen: 29 h 24** aktive Arbeitszeit über 17 Tage (Pausen bis 20 min als Arbeit gewertet; mit 45-min-Grenze: 35 h 44).
- Meine Summe aus Liste A war ~28 h — also insgesamt gut geschätzt, mit einer deutlichen Ausnahme: **den Seite-für-Seite-Vergleich habe ich um ~3 h unterschätzt**. Das ist genau der Posten, der am Ende der größte war.

B — Wie ich das nächste Mal vorgehen würde (empfohlen)

- **Erster Auftrag:** Template für den perfekten ersten Prompt, das viele Phasen zusammenfasst — liegt bei mir bereit, muss nur mit den Projektinformationen gefüllt werden.
- Korrektur, falls nötig.
- Neue Features und ggf. wieder Korrektur.
- **Vorschlag zur Ergänzung:** Aus dem tatsächlichen Verlauf fehlt hier ein Schritt, der garantiert kommt — **Kundenreview + Korrektur-Batch** (im Projekt Schritt 9 und 12, zusammen ~4 h). Den würde ich fest einplanen statt hoffen, dass er entfällt.

[Anmerkung: Liste B ist ein Plan für die Zukunft. Aus git log lässt sie sich nicht rekonstruieren — die Historie zeigt nur, was war, nicht was ich künftig vorhabe. Deshalb hier nur der eine belegbare Zusatz.]

Warum B schneller ist

- Der erste Prompt ist so genau, präzise und vollständig (~700 Zeilen), dass er fast alle Rückfragen vorwegnimmt.
- Sinngemäß: „Bau mir das Duplikat dieser Website und verbessere sie“ — nur eben sehr viel genauer.
- **Trotzdem** muss danach alles geprüft, korrigiert, ergänzt und perfektioniert werden. Der erste Prompt spart die Anläufe, nicht die Sorgfalt.

Prompt-Bibliothek

- Alle Prompts sind in den Sitzungstranskripten gespeichert.
- Ehrlich: als Bibliothek sind sie **wenig wert**, weil jeder seinen Prompt ohnehin mit KI-Hilfe auf sein Projekt zuschneiden sollte — so habe ich es auch gemacht.
- Der **erste Prompt** (das Template) liegt vor und ist der einzige, der sich wirklich wiederverwenden lässt.
- Eine detaillierte Zusammenfassung des Verlaufs steht in `docs/arbeitsbericht.md`.

Zwei Prompts, die tatsächlich versagt haben [ergänzt – Willy fragt ausdrücklich nach den gescheiterten]

- **„Bau die E-Rechnung-Seite nach“** (25.08.) — Ergebnis: Texte, Struktur und Farben stimmten, aber **ohne Animationen und ohne die Hintergrund-Atmosphäre**. Meine Rückmeldung: „*du hast nur die Texte kopiert und daraus eine rohe HTML-Datei gemacht.*“ Ursache und Lehre → Abschnitt 4.
- **„Setze den kompletten Design-Verbesserungsplan in einem Batch um“** (30.08.) — technisch sauber ausgeführt, sah aber **schlechter aus als vorher**. Komplette verworfen. ~1,5 h Totalverlust.

Do's & Don'ts

Was Claude sofort versteht

- Alles, was **eindeutig und klar** formuliert ist.
- Kleine, isolierte Aufgaben („diese eine Karte“, „dieser eine Button“).
- Wenn er zweifelt, soll er fragen → deshalb an jedes Auftragsende: „**Ask if something is not clear.**“

Womit das Tool überfordert ist

- „Bau mir eine Website, mach keinen Fehler.“ → kein Kontext, keine Regeln, kein Maßstab. Genau dagegen existiert `CLAUDE.md`.
- Alles, wo der Maßstab nur im Kopf des Auftraggebers liegt (Geschmacksfragen) — das muss man sehen, nicht beschreiben.

DO

- KI bei der Erstellung der Prompts mithelfen lassen.
- `CLAUDE.md` und der erste Prompt müssen **perfekt** sein.
- Claude committen lassen, aber **pushen nur auf ausdrückliche Erlaubnis** — dann muss man bei einem Fehlgriff nur lokal einen Commit löschen.
- Claude fragen, ob die Website den 8 Grundsätzen einer guten Website folgt: *point of view, typography that does work, a restrained color system, hierarchy that breathes, imagery with intent, motion that whispers, mobile that's designed not shrunk, the invisible expensive stuff*.
- Modus und Modell bewusst wählen.

DON'T

- Prompts unbedacht direkt in Claude Code tippen.
- Keine Ahnung davon haben, was man da eigentlich macht.

Refactoring vs. Scratch

- **Duplikation der Website:** alter Code wurde als Vorlage recycelt — Struktur, Texte, Farbwerte aus dem Original übernommen.
- **Neue Features (DRE):** komplett neu generiert.
- Auch beim recycelten Teil gab es viele kleine Anpassungen.
- **Wichtige Einschränkung** [ergänzt aus dem Projektverlauf]: „recycelt“ heißt hier **nicht** WordPress-/Elementor-Code übernommen. Der Elementor-Output ist unbrauchbar (1 363 KB gebündeltes CSS, davon nutzt die Seite einen Bruchteil). Recycelt wurden **Inhalt und gemessene Design-Werte**, neu geschrieben wurde das gesamte Markup und CSS. Ergebnis: 126 KB statt 1 363 KB.

Noch offen (Abschnitt 2)

- **Prompts in Kategorien aufteilen** — Willy schlägt „Struktur aufbauen / Design/CSS nachbauen / Inhalte befüllen“ vor. Diese Kategorien passen auf unseren Ablauf **nicht**, weil so nicht gearbeitet wurde (siehe Arbeitszyklus). Eine passende eigene Kategorisierung habe ich noch nicht geschrieben.

3. Revisionen & Workflow

Inhaltliche Anpassungen (Texte/Bilder tauschen)

- **Deutlich schneller als im WordPress-Backend.** Größenordnung 15–30 min für eine normale Änderungsrunde.
- Änderungen lassen sich **als Batch** übergeben — eine Nachricht, mehrere Punkte.
- Claude tauscht nicht nur das Bild, sondern **passt das Design gleich mit an** (Größe, Form, Umbrüche drumherum).
[gemessen: Willys zweiter Batch am 09.09. — 9 Punkte auf einmal (zwei Textkorrekturen, zwei Rollen, ein neues Foto, eine fertiggestellte Karte, ein CSS-Bug site-weit, Navigation umgebaut, Logo-Größen, eine Sektion verkleinert) = **1 h**]
- **20 inklusive Tests und Sichtprüfung.** Reine Textkorrekturen darin: wenige Minuten.]
- **Gegenrechnung WordPress:** ein einzelner Texttausch im Backend ist per Klick schneller als ein Prompt. Der Vorteil kippt, sobald es **mehrere Änderungen auf einmal** sind oder die Änderung **Layout-Folgen** hat — dann macht Claude in einem Durchgang, wofür im Backend viele Klicks plus Nacharbeit am Theme nötig wären.

Visuelles Finetuning (CSS-Details)

- Ja, Claude Code beherrscht das sehr gut.
- [Belege aus dem Projekt — Willy fragt konkret nach Spacings, Breakpoints und Hover-Effekten:]
 - **Spacings** — „die Abstände nach den CTA-Bändern sind zu groß“: 156 → 116 px, 172 → 116, 216 → 159. Umgesetzt über eine Regel, die auf die *Form* der Sektion greift, nicht auf die zwei namentlich genannten Stellen.
 - **Mobile Breakpoints** — Audit über 12 Seiten × 6 Handybreiten (320–430 px); gefunden und behoben: 6 px horizontaler Seitenscroll auf `/kontakt/` bei 320 px, plus Wortumbrüche mitten im Wort auf sechs Seiten.
 - **Hover-Effekte** — sechs Mikrointeraktionen, bewusst dezent. Eine musste nachgebessert werden: „*I can't see the cursor glow*“ — zu schwach eingestellt und nur auf einer Seite ausgespielt.
- **Grenze:** Claude kann jeden gewünschten Wert exakt umsetzen, aber **nicht entscheiden, welcher Wert gut aussieht**. Das musste jedes Mal ich beurteilen.

Versionskontrolle (Git)

- **Commits macht Claude selbst**, nach jeder abgeschlossenen logischen Änderung — nicht nach jedem einzelnen Schritt. [gemessen: 86 Commits]
- **Push auf GitHub nur mit meiner ausdrücklichen Erlaubnis.** Grund: solange nichts gepusht ist, reicht es, einen Commit **lokal** zu löschen, wenn etwas schiefgeht.
- Diese Regel steht in `CLAUDE.md` und wurde jedes Mal eingehalten — im Projekt ist sie zweimal wertvoll geworden:
 - **30.08.:** der verworfene Design-Versuch — ein Commit gelöscht, Stand wieder sauber, nichts war je öffentlich.
 - **08.09.:** zwei Dateien mit meinem eigenen Arbeitsmaterial lagen im Repo, kurz bevor gepusht wurde. Da das GitHub-Repo **öffentlich** ist, wären sie dauerhaft in der Historie gelandet. Stattdessen: Dateien raus, die drei betroffenen Commits aus dem Branch entfernt, dann erst gepusht.
- **Kontrolle behalten:** ein Commit pro logischer Änderung, aussagekräftige Commit-Messages, und `PROGRESS.md` als datiertes Protokoll mit Commit-Hashes.

Noch offen (Abschnitt 3)

- Keine offenen Punkte — die drei Fragen aus Willys Briefing sind beantwortet.

4. Bottlenecks & Grenzen

Ungeschminkt. Das ist der Teil, der die Fallstudie belastbar macht.

Context Window & Vergesslichkeit

- **11 Kompaktierungen im Projekt.** Jede längere Sitzung lief voll; danach hatte Claude nur noch eine Zusammenfassung statt des echten Verlaufs.
- **Folge:** die nächste Sitzung begann faktisch bei null. Deshalb wurde am 25.08. `PROGRESS.md` eingeführt — ausdrücklich **nicht für mich, sondern für Claude:** „*Not for me, but for Claude to keep track of what we've done so far.*“
- Meine erste Fassung davon wurde von mir selbst verworfen, weil Claude Projektziel und Regeln aus `CLAUDE.md` wiederholt hatte → Doppelung kostet nur Tokens. Endstand: reines datiertes Was-wurde-getan-Protokoll, am Ende **859 Zeilen**.
- **Hat Claude bestehende Styles überschrieben?** Nicht durch Vergesslichkeit, aber durch zu breite Regeln — z. B. eine Regel, die für die Startseite gedacht war, aber auf die **geteilte Kopfleiste** aller 12 Seiten durchschlug, sodass derselbe Button auf der Startseite eckig und überall sonst rund war (08.09., `ede6c0c`).
- **Preis:** Zeit in jeder Sitzung fürs Wiedereinlesen. Kein Randproblem, sondern der Grund für eine eigene Datei.

Bugs, Sackgassen, manuelle Eingriffe

1. Vercel-Deployment schlug fehl — Root Directory falsch (23.08.)

- **Was ich wollte:** erstes Deployment der fertig gebauten Seiten.
- **Was rauskam:**

```
Built 11 pages:
dist/404.html
dist/e-rechnung/index.html
... (alle 11 Seiten korrekt gebaut) ...
Error: No Output Directory named "dist" found after the Build completed.
Configure the Output Directory in your Project Settings.
```

- **Warum das schwer zu finden war:** Das Build-Log sah **erfolgreich** aus — `dist/` wurde nachweislich mit allen 11 Seiten erzeugt. Trotzdem behauptete Vercel, es gebe kein `dist`. `vercel.json` sagte korrekt `"outputDirectory": "dist"`.
- **Erster Fehlversuch:** ich habe im Vercel-Dashboard von Hand Output Directory auf `.` und Build Command auf `echo` gesetzt → machte es schlimmer, weil das Projekt jetzt teils überschriebene, teils originale Felder hatte.
- **Diagnose:** Claude ließ mich die Projekteinstellungen vorlesen statt weiter zu raten. Ergebnis: **Root Directory stand auf `src/pages` statt leer.**
- **Warum das genau diesen Fehler erzeugt:** Vercel nimmt Root Directory als Basis für alles. Der Build lief trotzdem durch (`build.js` schreibt nach `dist/` im echten Repo-Root, weil die Datei dort liegt) — aber gesucht wurde danach unter `src/pages/dist`, und das existiert nicht.
- **Fix:** Root Directory auf leer (Repo-Root), Dashboard-Overrides zurück auf die Werte aus `vercel.json`. Danach lief jeder Push automatisch durch.
- **Lehre:** Ein grünes Build-Log heißt nicht, dass das Deployment funktioniert. Und: Konfiguration an zwei Orten (Dashboard **und** `vercel.json`) ist eine Fehlerquelle — einer davon sollte führen.

2. Der 1:1-Irrtum — der teuerste Fehler des Projekts (25.–26.08.)

- **Was ich wollte:** die E-Rechnung-Seite 1:1 wie das Original.
- **Was rauskam:** Struktur, Texte und Farben stimmten — aber die Seite wirkte wie ein rohes HTML-Dokument. Meine Rückmeldung: „*du hast nur die Texte kopiert und keine Animationen gemacht.*“
- **Ursache (methodisch, nicht handwerklich):** Claude hatte die Seite aus **DOM-Struktur und berechneten Stilwerten** rekonstruiert. Dabei geht systematisch verloren:
 - **Bewegung** — das Original blendet fast jedes Element beim Scrollen ein (Elementor `fadeInUp`, `fadeInLeft`). Im Quelltext steht davon nichts Sichtbares: die Elemente parken auf Deckkraft 0.
 - **Atmosphäre** — drei Bänder stehen auf Schwarz mit türkis-blauem Leuchten. Aus den berechneten Stilen war nur „schwarz + Hintergrundbild“ ablesbar; die Annäherung mit einem gedämpften Verlauf traf die Wirkung nicht. Das musste ich **ein zweites Mal** einzeln monieren.
- **Fix:** Regel geändert auf „*1:1 heißt, was der Nutzer sieht, nicht was im Code steht.*“ Ab da: Originalseite mit Playwright fahren, scrollen wie ein Besucher, an mehreren Breiten Screenshots, Band für Band vergleichen.
- **Preis:** kompletter Neubau der Seite (~2 h) — und dasselbe Muster tauchte später bei Startseite und Ergebnisse noch einmal auf.

3. Inhaltsdrift gegen die „1:1, nichts erfinden“-Regel

- **Startseite (28.08.):** Auftrag war ein Deep-Audit, weil die Seite „*inhaltlich und im Design zu weit abgedriftet*“ war. Komplett gegen das Original neu aufgebaut (`60fb6e2`), mit ausdrücklicher Regel Nr. 1: nichts erfinden.
- **E-Rechnung (05.09.):** Willys Review kehrte **drei meiner eigenen früheren Entscheidungen um** — die Seite sollte ohne jede Abweichung dem Original entsprechen. Messung vorher: 56 Textblöcke wichen in der Schriftfamilie ab, 17 in der Größe, 30 in der Zeilenhöhe. Nachher: **null** Abweichung in Größe, Gewicht und Farbe.
- **DRE-Texte (27.08.):** Willy lieferte den finalen Text mit der Ansage „*word for word — do not rewrite, paraphrase, summarize, restructure or 'fix' any of it*“, ausdrücklich **inklusive dem, was wie ein Tippfehler aussieht**. Vorher hatte Claude aus drei überlappenden Entwürfen eine eigene Fassung gebaut — das war beauftragt, aber es zeigt: ohne die ausdrückliche Wortlaut-Regel formuliert das Tool um.
- **Lehre:** Die Regel „nichts erfinden“ muss **in jedem einzelnen Auftrag** stehen, nicht nur einmal in `CLAUDE.md`.

4. Falsche Zielseite beim Antippen — nur auf echtem Gerät auffindbar (03.09.)

- **Symptom (von mir gemeldet):** „*Mit dem Handy lande ich beim Klick auf 'Vertraulichkeit' auf der falschen Seite, manchmal auf der richtigen.*“
- **Ursache:** das geschlossene Navigationsmenü lag unsichtbar über der Seite und fing Berührungen ab. `visibility` wird vererbt, und ein Kind darf das `hidden` des Elternteils mit eigenem `visible` überschreiben — genau das passierte bei den vier Untermenü-Links.
- **Warum „manchmal“:** es hing davon ab, wie weit die Seite gescrollt war.
- **Am Schreibtisch nicht reproduzierbar.** Ohne echtes Telefon wäre das live gegangen.

5. Nicht reproduzierbare Testergebnisse

- Ein Prüfskript meldete bei **identischem Code** in aufeinanderfolgenden Läufen unterschiedliche Ergebnisse (einmal 13 Funde / 8 verschiedene, dreimal 7 / 2).
- Ursache: Zeitabhängigkeit beim Laden von Bildern.
- Auch am 10.09. wieder: ein Lauf meldete abgeschnittenen Text im rotierenden Wort der Startseite, der nächste war sauber — das Element ändert während seiner Animation die Breite.
- **Konsequenz als Dauerregel:** bei einem überraschenden Sprung **erst wiederholen, dann glauben**.

6. Windows-Eigenheiten

- Zeilenenden (CRLF) und Zeichenkodierung haben mehrfach Dateien beschädigt — einmal einen Git-Patch komplett unbrauchbar gemacht.
- `python.exe` nicht im PowerShell-PATH.
- Windows unterscheidet keine Groß-/Kleinschreibung bei Dateinamen, **Vercel schon**: ein Ordner `DRE` statt `dre` hätte in Produktion eine ganze Sektion auf 404 laufen lassen. Lokal fiel es nicht auf.
- Alles drei ist inzwischen als Dauerregel dokumentiert, damit es nicht erneut Zeit kostet.

7. Die Vorlage selbst ist defekt — nicht unser Bug, aber Aufwand

- **Alle** Unterseiten-Permalinks der Live-Seite liefern 404 und antworten nur unter `/index.php/<slug>/` (WordPress-Rewrite kaputt).
- Das Kundenlogo-Karussell liefert kaputte Bilder.
- Ein Band hat seinen Hintergrund verloren → **weiße Überschrift auf weißem Grund, unsichtbar**.
- **Vier von zehn Seiten scrollen am Handy seitlich** (gemessen 10.09. bei 390 px Viewport): Kommunikation 476 px breit, Kontakt 421, Automatische Rechnungsprüfung 411, Startseite 400. Der Neubau liegt auf allen zehn Seiten exakt bei 390.
- **Erzwang eine Grundsatzentscheidung**: Wie weit reicht „1:1“, wenn das Original kaputt ist? Meine Regel: *offensichtlich kaputt wird nicht mitkopiert, bewusste Inkonsistenz schon*. Testfrage: weißer Text auf verschwundenem Hintergrund = Bug. Zwei Karten mit verschiedenen Überschriftgrößen = Entscheidung. **Hässlich ist nicht dasselbe wie kaputt**.

8. Nutzungslimits mitten in der Arbeit

- Dreimal wurde eine laufende Aufgabe hart unterbrochen („*can you resume where you were interrupted? The limit barrier should be gone now*“).
- Danach jeweils 5 h Wartezeit oder Wechsel auf ein schwächeres Modell.
- **Nicht planbar** — und der einzige Punkt, an dem das Werkzeug den Arbeitsfluss wirklich diktiert hat.

9. Wo Claude gefragt hat, statt zu raten — und warum das gut war

- **23.08., mitten in Stufe 1**: „*Ich bin auf etwas gestoßen, das einen Teil deiner Freigabe hinfällig macht, und will deine Entscheidung, bevor ich baue.*“ Anlass war die Entdeckung der kaputten Permalinks.
- **10.09., Wortumbruch**: ein Rollenwort brauchte 167 px, die Spalte hatte 145. Statt eigenmächtig die Schrift zu verkleinern → drei Optionen vorgelegt, ich habe entschieden (Kartenabstand kürzen).
- **09.09., vor dem Push**: Claude hat gemeldet, dass das GitHub-Repo öffentlich ist, **bevor** zwei Dateien mit meinem eigenen Arbeitsmaterial hochgeladen wurden. Das hätte sich nachträglich nicht sauber rückgängig machen lassen.
- **Muster**: Bei allem, was **Aussehen** oder **Außenwirkung** verändert, wurde gefragt. Bei reinen Darstellungsfehlern wurde ohne Rückfrage korrigiert. Diese Trennung stand ab dem 24.08. in `CLAUDE.md` und hat bis zum Ende gehalten.

Token- und Kostenaufwand

- **Abo**: Claude Pro, 20 €/Monat. Kein zusätzlicher API-Verbrauch.
- **Reichweite**: ca. 2,5–3 h Arbeit mit Opus auf Max Effort, dann 5 h Sperre. Im Projekt dreimal erreicht.
- **Teuerste Posten** (nach Gefühl, keine Token-Messung verfügbar):
 1. **Seite-für-Seite-Vergleich** (~10 h) — viele Playwright-Läufe, viele Screenshots, jeder Vergleich zieht Bilddaten in den Kontext.

2. **Der Neubau nach dem 1:1-Irrtum** — dieselbe Seite zweimal gebaut.

3. **Kontextverluste** — nach jeder Kompaktierung muss `PROGRESS.md` neu gelesen werden.

- **Billigste Posten:** die eigentliche Code-Erzeugung. Zehn Seiten in gut zwanzig Minuten.
 - **Einordnung:** Bei 29 h 24 aktiver Arbeit und 20 €/Monat liegen die reinen Werkzeugkosten des Projekts **unter 20 €** — das Abo lief ohnehin. Der Aufwand steckt in der **Arbeitszeit**, nicht in der Lizenz.
 - **Grenze der Aussage:** Claude Code zeigt keinen Token-Zähler pro Aufgabe. Alle Angaben hier sind Beobachtung, keine Messung.
-

5. Fazit & B2B-Vergleich

Vergleichstabelle WordPress vs. Claude Code

Kriterium	WordPress (alt)	Claude Code / statisch (neu)
Setup bis erste Live-Seite	Branchenüblich ~2–5 h (<i>Schätzung: Hosting + WP-Installation + Theme + Grundkonfiguration; nicht in diesem Projekt gemessen</i>)	~3 h gemessen (Setup 1 h + Planung 1,5 h + Deployment 0,7 h) — der Nachbau aller 10 Seiten kam mit 20 min obendrauf
Gesamtaufwand Migration	—	29 h 24 aktiv, 17 Tage, inkl. Recherche, Kundenreview und Dokumentation
Lighthouse Performance — Handy	62	91
Lighthouse Performance — Desktop	90	97
Lighthouse Accessibility	92	96
Lighthouse Best Practices / SEO	100 / 100	100 / 100
Largest Contentful Paint — Handy	6,9 s	2,8 s
Largest Contentful Paint — Desktop	1,2 s	1,0 s
First Contentful Paint — Handy	5,3 s	2,6 s
Cumulative Layout Shift (Desktop)	0,142	0,001
Total Blocking Time (Desktop / Handy)	20 ms / 140 ms	0 ms / 60 ms
Server-Antwortzeit	160 ms	10 ms
Seitengewicht (Desktop / Handy)	875 / 1 184 KiB	256 / 256 KiB
Main-Thread-Arbeit (Desktop)	3,5 s	0,4 s
JavaScript-Ausführung (Desktop)	0,5 s	0,0 s
Seiten mit seitlichem Scroll am Handy	4 von 10	0 von 10
CSS-Auslieferung	1 363 KB gebündelt (Elementor), Seite nutzt einen Bruchteil	126 KB

Kriterium	WordPress (alt)	Claude Code / statisch (neu)
Flexibilität bei Designwünschen	An Theme + Page-Builder gebunden; Sonderwünsche brauchen Child-Theme, Plugin oder Custom CSS	Jede Änderung direkt im Quelltext; Sonderwünsche kosten so viel wie das Beschreiben
Wartung & Sicherheit	Core-, Theme- und Plugin-Updates dauerhaft nötig; jedes Plugin ist Angriffsfläche	Null Abhängigkeiten , keine Datenbank, kein PHP — nichts, was veralten oder gekapert werden kann
Rückabwicklung von Fehlern	Backup/Staging nötig	Ein Commit löschen
Automatisierte Qualitätsprüfung	Nicht vorhanden	Testskript prüft 11 Seiten × 5 Breiten auf Überläufe, abgeschnittene Texte, tote Links, Konsolenfehler
Redaktion durch den Kunden	GUI im Backend, jeder kann Texte ändern	Nicht möglich ohne Entwickler — der größte Nachteil

Messgrundlage Lighthouse: Lighthouse 13.4.1, 10.09.2026. Zwei Läufe je Seite — `--preset=desktop` und der Mobile-Standard (gedrosselte CPU und Netzwerk, wie Google es für die Suchbewertung ansetzt).

Ehrliche Einordnung der Zahlen

- **Am Desktop ist WordPress nicht katastrophal** — 90 Performance, 100 bei Best Practices und SEO. Wer das Gegenteil behauptet, verkauft. Der Unterschied liegt dort bei **Stabilität und Leerlauf**: CLS 0,142 → 0,001, Main-Thread 3,5 s → 0,4 s, JavaScript 0,5 s → 0,0 s. Der erste Bildaufbau ist identisch (FCP 1,0 s bei beiden).
- **Am Handy fällt die Entscheidung.** 62 gegen 91 Punkte, LCP **6,9 s gegen 2,8 s**. Das ist der Wert, den Google am stärksten gewichtet, und 6,9 s liegt weit jenseits der 2,5-s-Schwelle für „gut“. Auf dem Gerät, mit dem die meisten Besucher kommen, ist der Unterschied also kein Feinschliff, sondern zweieinhalbfach.
- **CLS ist ein Core Web Vital.** Am Desktop 0,142 (über der 0,1-Schwelle) gegen praktisch null. Am Handy sind beide bei 0.
- **Vier von zehn Seiten des Originals lassen sich am Handy seitlich schieben** — gemessen bei 390 px Viewport: Kommunikation 476 px, Kontakt 421, Automatische Rechnungsprüfung 411, Startseite 400. Alle zehn Seiten des Neubaus sind exakt 390. Belege als Screenshot-Paare in `docs/baseline/`.
- **Zwei Werte sind nicht besser:** Speed Index am Desktop 1,1 s (alt) gegen 1,2 s (neu), praktisch gleich; und die Accessibility-Differenz (92 → 96) ist eine Verbesserung, aber keine, die ein Kunde spürt. Gehört trotzdem genannt.

Hosting-Kosten (IONOS, Stand 10.09.2026)

	WordPress	Statische Seite
Tarif	WordPress Hosting Start	Webhosting Standard (<i>kein WordPress-Tarif nötig — ohne Datenbank und CMS</i>)
Aktionspreis	3 €/Monat, 6 Monate	3 €/Monat, 6 Monate
Regulär danach	5 €/Monat	6 €/Monat
Speicher	25 GB	100 GB NVMe
Domain / Postfach	1 / 1	1 / 1

- **Ergebnis ehrlich: beim reinen Hosting-Preis gibt es keinen Vorteil.** Der passende Nicht-WordPress-Tarif ist regulär sogar **1 €/Monat teurer** — er bringt dafür die vierfache Speichermenge, die eine statische Seite nicht braucht.
- **Claude Pro (20 €/Monat) gehört nicht in diese Rechnung.** Das ist ein allgemeines Entwickler-Abo, kein Hosting-Posten pro Website — es läuft unabhängig davon, wie viele Kundenprojekte darauf gebaut werden.
- **Wo der Kostenvorteil tatsächlich liegt:**
 1. **Setup-Zeit** — der Nachbau selbst dauerte 20 Minuten.
 2. **Keine kostenpflichtigen Premium-Themes oder -Plugins.**
 3. **Laufende Wartung** — keine Update-Zyklen, keine Plugin-Konflikte, keine Kompatibilitätstests nach jedem WP-Core-Release.
 4. **Statisches Hosting ist auch kostenlos zu haben** — dieses Projekt läuft auf Vercels kostenloser Stufe. Die IONOS-Zeile oben ist der Vergleich, den Willy angefragt hat, nicht das, was hier tatsächlich bezahlt wird.

Persönliches Fazit

- **Ja, ich würde Claude Code für ein Kundenprojekt wieder wählen.**
- **Gründe:** schneller und einfacher zu benutzen als WordPress, flexibler bei Änderungen.
- **Aber:** wir haben **weniger Kontrolle** als bei WordPress.
- **Für welchen Kundentyp:** heute eignet sich Claude Code aus meiner Sicht für **alle** Kundentypen. Entscheidend ist der Mensch dahinter — er muss den Tech-Stack verstehen.
 - Kunde mit rein statischer Website wie KIBH → **Basis-Kenntnisse in Webentwicklung reichen.**
 - Website mit mehr Backend → entsprechend mehr Verständnis nötig.
- Hier war die Aufgabe eine **Web-Migration**, d. h. viel Bestehendes wurde wiederverwendet.

Weitere Punkte (Vorschlag — von Claude ergänzt, nicht meine ursprünglichen Gründe)

Zusätzliche Vorteile, technisch belegbar

- **Kleinere Angriffsfläche.** Kein PHP, keine Datenbank, keine Plugins. Die häufigste WordPress-Einbruchsursache — ein veraltetes Plugin — existiert nicht.
- **Kein Anbieter-Lock-in.** Die Seite ist ein Ordner mit HTML, CSS und Bildern. Sie läuft auf jedem Webspace, jedem CDN, jedem S3-Bucket. Ein Umzug ist ein Kopiervorgang.
- **Messbar bessere Kernwerte statt Gefühl** — die Lighthouse-Zahlen oben sind reproduzierbar und lassen sich einem Kunden vorlegen.

- **Migration deckt Altlasten auf.** Beim Nachbau fielen drei echte Defekte der Live-Seite auf, die dem Kunden niemand gemeldet hatte: 404 auf allen Unterseiten-Permalinks, kaputte Logo-Bilder, unsichtbare Überschrift. Das ist ein Verkaufsargument für sich.
- **Prüfbarkeit als Liefergegenstand.** Das Testskript kann bei jeder späteren Änderung erneut laufen. Bei WordPress gibt es diese Zusicherung nicht.

Zusätzliche Nachteile, ehrlich

- **Der Kunde kann Inhalte nicht selbst pflegen.** Kein Backend, keine GUI. Jede Textänderung braucht jemanden mit Claude Code. **Das ist der größte Nachteil gegenüber WordPress** und muss im Angebot stehen — entweder als Wartungsvertrag oder mit einem späteren, schlanken CMS.
 - **Die Qualität hängt an einem Menschen, der hinsieht.** 33 Screenshots und praktisch jede Korrektur kamen daher, dass ich die Seite im Browser beurteilt habe. Zwei Fehler waren nur so auffindbar. **Ohne diese Prüfschleife wäre die Seite fertig und falsch gewesen.**
 - **Prompt-Qualität ist Projektqualität.** Präzise Aufträge lieferten brauchbare Ergebnisse, vage erzeugten Nacharbeit. Diese Fähigkeit muss man haben oder einkaufen.
 - **Nicht belegt durch dieses Projekt:** Verhalten bei umfangreicher Serverlogik, bei mehreren hundert Seiten, oder bei Redaktionsteams. Sollte in der Fallstudie ehrlich als offen benannt werden.
-

Quellen und Nachweise

Nachweis	Ort
Detaillierter Arbeitsbericht, 13 Arbeitsschritte	<code>docs/arbeitsbericht.md</code>
Datiertes Sitzungsprotokoll mit Commit-Hashes	<code>PROGRESS.md</code>
Aufgabenstellung und Projektregeln	<code>CLAUDE.md</code>
Visuelle Baseline alt/neu, 10 Seiten × 2 Breiten	<code>docs/baseline/</code>
Messwerte Seitengewicht, Methodik, Vorbehalte	<code>docs/performance-baseline.md</code>
Design-Werte mit Fundstelle im Original	<code>docs/design-system.md</code>
URL-Zuordnung alt → neu, inkl. 404-Befund	<code>docs/url-map.md</code>
Commit-Historie	<code>git log</code> — 86 Commits, 23.08.–10.09.2026
Lighthouse-Rohdaten	reproduzierbar mit <code>npx lighthouse <url> --preset=desktop</code>
IONOS-Preise	ionos.de/hosting/webhosting · ionos.de/hosting/wordpress-hosting